Lab 3: Problem ograniczonego bufora (producentów-konsumentów)

Oliwia Dyszewska

listopad 2025

1 Opis zadania

Dany jest bufor, do którego producent może wkładać dane, a konsument pobierać. Napisać program, który zorganizuje takie działanie producenta i konsumenta, w którym zapewniona będzie własność bezpieczeństwa i żywotności. Zrealizować program - przy pomocy metod wait()/notify(). Patrz listing 1.

1. dla przypadku 1 producent/1 konsument
2. dla przypadku n_1 producentów / n_2 konsumentów ($n_1 > n_2$; $n_1 = n_2$; $n_1 < n_2$)
3. wprowadzić wywołanie metody sleep() i wykonać pomiary, obserwując zachowanie producentów/konsumentów

1.1 Przetwarzanie potokowe z buforem

1. Bufor o rozmiarze N
2. Proces A bedący producentem.
3. Proces Z bedący konsumentem.
4. Procesy B, C, . . . , Y bedące procesami przetwarzającymi. Każdy proces otrzymuje daną wejściową od procesu poprzedniego, jego wyjście zaś jest konsumowane przez proces następny.
5. Procesy dzialają z różnymi prędkościami.

2 Opis programu

2.1 Producer.java

Produkuje kolejne liczby całkowite i odkłada je do bufora aż do osiągnięcia zadanego czasu końcowego.

```
public class Producer extends Thread {
    private final Buffer buf;          // bufor
    private final int sleepMs;         // pauza miedzy produkcjami (do
        obserwacji)
    private final long endTimeMs;      // znacznik czasu zakonczenia pracy
    private long produced = 0;         // statystyka ile elementow wyprodukowano

    public Producer(Buffer buf, int sleepMs, long endTimeMs, String name) {
        super(name);
        this.buf = buf;
        this.sleepMs = sleepMs;
```

```java
        this.endTimeMs = endTimeMs;
    }

    public long getProduced() { return produced; }

    @Override public void run() {
        int x = 0;
        while (System.currentTimeMillis() < endTimeMs) {
            buf.put(x++);
            produced++;
            if (sleepMs > 0) {
                try { Thread.sleep(sleepMs); } catch (InterruptedException
                    ignored) {}
            }
        }
    }
}
```

2.2 Consumer.java

Pobiera elementy z bufora aż do napotkania wartości „trucizny” (POISON), po której kończy działanie.

```java
public class Consumer extends Thread {
    private final Buffer buf;       // bufor
    private final int sleepMs;      // pauza miedzy konsumpcjami
    private final int poison;       // specjalna wartosc sygnalizujaca koniec
    private long consumed = 0;      // statystyka ile elementow skonsumowano

    public Consumer(Buffer buf, int sleepMs, int poison, String name) {
        super(name);
        this.buf = buf;
        this.sleepMs = sleepMs;
        this.poison = poison;
    }

    public long getConsumed() { return consumed; }

    @Override public void run() {
        while (true) {
            int v = buf.get();
            if (v == poison) break; // trucizna konczy prace konsumenta
            consumed++;
            if (sleepMs > 0) {
                try { Thread.sleep(sleepMs); } catch (InterruptedException
                    ignored) {}
            }
        }
    }
}
```

2.3 Buffer.java

Ograniczony bufor zaimplementowany przy wait() i notifyAll().

```java
class Buffer {
    private final int[] data;    // tablica przechowujaca dane
    private int head = 0;        // indeks odczytu
    private int tail = 0;        // indeks zapisu
    private int count = 0;       // aktualna liczba elementow w buforze

    public Buffer(int capacity) {
        if (capacity <= 0) throw new IllegalArgumentException();
        this.data = new int[capacity];
    }

    // Wklada element do bufora, jesli jest pelny to czeka.
    public synchronized void put(int i) {
        while (count == data.length) {
            try { wait(); } catch (InterruptedException ignored) {}
        }
        data[tail] = i;
        tail = (tail + 1) % data.length;
        count++;
        notifyAll();
    }

    // Pobiera element z bufora, jesli jest pusty to czeka.
    public synchronized int get() {
        while (count == 0) {
            try { wait(); } catch (InterruptedException ignored) {}
        }
        int i = data[head];
        head = (head + 1) % data.length;
        count--;
        notifyAll();
        return i;
    }

    public int capacity() { return data.length; }
}
```

2.4 PKmon.java

Klasa główna:

- konfiguruje liczbę producentów/konsumentów i parametry bufora,
- startuje wątki,
- po zakończeniu producentów wstawia do bufora „trucizny” (po 1 na konsumenta),
- zbiera i wypisuje statystyki oraz czas wykonania.

```java
public class PKmon {
    private static final int POISON = Integer.MIN_VALUE; // wartosc
        nieprodukowana przez producentow

    public static void main(String[] args) {
        int nProd   = 1;     // liczba producentow (n1)
```

```java
int nCons    = 1;     // liczba konsumentow (n2)
int capacity = 10;    // pojemnosc bufora
int pSleepMs = 1;     // pauza producenta (ms)
int cSleepMs = 1;     // pauza konsumenta (ms)
int durationSeconds = 2; // czas pracy producentow (s)

Buffer buf = new Buffer(capacity);

// Dla czytelnego wypisu relacji n1 z n2
String rel = nProd == nCons ? "=" : (nProd > nCons ? ">" : "<");

long endTime = System.currentTimeMillis() + durationSeconds * 1000L;

Producer[] producers = new Producer[nProd];
Consumer[] consumers = new Consumer[nCons];

// Tworzymy watki
for (int i = 0; i < nProd; i++) {
    producers[i] = new Producer(buf, pSleepMs, endTime, "Producer-" +
        i);
}
for (int i = 0; i < nCons; i++) {
    consumers[i] = new Consumer(buf, cSleepMs, POISON, "Consumer-" +
        i);
}

long t0 = System.currentTimeMillis();
for (Thread t : producers) t.start();
for (Thread t : consumers) t.start();

// Czekamy, az wszyscy producenci zakoncza
try {
    for (Thread t : producers) t.join();
} catch (InterruptedException ignored) {}

// Wstawiamy trucizny
for (int i = 0; i < nCons; i++) {
    buf.put(POISON);
}

// Czekamy, az konsumenci skoncza po zjedzeniu trucizny
try {
    for (Thread t : consumers) t.join();
} catch (InterruptedException ignored) {}

long t1 = System.currentTimeMillis();

long totalProduced = 0, totalConsumed = 0;

System.out.printf(
        "Wariant: %dP %s %dK  | Bufor=%d | pSleep=%dms | cSleep=%dms
            | czas=%ds%n",
        nProd, rel, nCons, capacity, pSleepMs, cSleepMs,
            durationSeconds
```

```
        );

        System.out.println("\n—— STATYSTYKI ——");
        for (Producer p : producers) {
            System.out.printf("%s: produced=%d%n", p.getName(),
                p.getProduced());
            totalProduced += p.getProduced();
        }
        for (Consumer c : consumers) {
            System.out.printf("%s: consumed=%d%n", c.getName(),
                c.getConsumed());
            totalConsumed += c.getConsumed();
        }

        System.out.printf("SUMA: produced=%d, consumed=%d%n", totalProduced,
            totalConsumed);
        System.out.printf("Czas wykonania: %d ms%n", (t1 - t0));
    }
}
```

2.5 Przetwarzanie potokowe z buforem

2.5.1 Stage.java

Czyta z bufora wejściowego, zapisuje do bufora wyjściowego. Propaguje dalej dokładnie tyle trucizn (POISON), ilu jest końcowych konsumentów,żeby na końcu każdy konsument dostał sygnał zakończenia.

```
class Stage extends Thread {
    private final Buffer in, out;          // kolejki: wejsciowa i wyjsciowa
    private final int POISON;              // specjalna wartosc sygnalizujaca
        koniec
    private final int sleepMs;             // predkosci etapow
    private final int poisonsToPass;       // ile trucizn trzeba przepuscic
        dalej
    private long processed = 0;            // licznik elementow

    Stage(Buffer in, Buffer out, int poison, int sleepMs, String name, int
        poisonsToPass) {
        super(name);
        this.in = in; this.out = out;
        this.POISON = poison; this.sleepMs = sleepMs;
        this.poisonsToPass = poisonsToPass;
    }

    public long getProcessed() { return processed; }

    public void run() {
        int passed = 0; // ile POISON juz przepuscilismy dalej
        while (true) {
            int v = in.get();
            if (v == POISON) {
                out.put(POISON);
                if (++passed == poisonsToPass) // konczymy dopiero po
                    przepchnieciu wszystkich
                    break;
```

```
                continue;
            }
            processed++;
            out.put(v);
            if (sleepMs > 0) {
                try { Thread.sleep(sleepMs); } catch (InterruptedException
                    ignored) {}
            }
        }
    }
}
```

2.5.2 PKmon2.java

Budujemy potok A->B->...->Z z K etapami pośrednimi. Między każdą parą etapów jest osobny, ograniczony bufor z wait()/notifyAll(). Każdy Stage przepuszcza dalej wszystkie POISON i dopiero wtedy kończy. Dzięki temu na końcu każdy konsument dostaje jedną truciznę i może się poprawnie domknąć.

```
public class PKmon2 {
    private static final int POISON = Integer.MIN_VALUE;

    public static void main(String[] args) {
        int nProd = 1, nCons = 2;
        int capacity = 10;
        int pSleepMs = 1, cSleepMs = 1;
        int durationSeconds = 5;

        // Konfiguracja potoku
        int K = 3; // liczba etapow posrednich: B,C,D

        int[] stageSleep = new int[] { 2, 0, 3 }; // rozne predkosci B,C,D

        // Bufory: A->B, B->C, ..., ostatni->Z
        Buffer[] q = new Buffer[K + 1];
        for (int i = 0; i < q.length; i++) q[i] = new Buffer(capacity);

        long endTime = System.currentTimeMillis() + durationSeconds * 1000L;

        // Producenci (A) -> q[0]
        Producer[] producers = new Producer[nProd];
        for (int i = 0; i < nProd; i++)
            producers[i] = new Producer(q[0], pSleepMs, endTime, "Producer-"
                + i);

        // Etapy B.. (q[i] -> q[i+1])
        Stage[] stages = new Stage[K];
        for (int i = 0; i < K; i++) {
            stages[i] = new Stage(q[i], q[i+1], POISON, stageSleep[i],
                    "Stage-" + (char)('B'+i), nCons);
        }


        // Konsumenci (Z) czytaja z q[K]
        Consumer[] consumers = new Consumer[nCons];
        for (int i = 0; i < nCons; i++)
```

```
            consumers[i] = new Consumer(q[K], cSleepMs, POISON, "Consumer-" +
                i);

        long t0 = System.currentTimeMillis();
        for (Thread t : producers) t.start();
        for (Thread s : stages) s.start();
        for (Thread t : consumers) t.start();

        for (Thread t : producers) try { t.join(); } catch
            (InterruptedException ignored) {}

        // Wstrzykujemy POISON do pierwszego bufora
        for (int i = 0; i < nCons; i++) q[0].put(POISON);

        for (Thread s : stages) try { s.join(); } catch (InterruptedException
            ignored) {}
        for (Thread t : consumers) try { t.join(); } catch
            (InterruptedException ignored) {}
        long t1 = System.currentTimeMillis();

        // STATYSTYKI
        long totalProduced = 0, totalConsumed = 0;

        System.out.printf(
                "POTOK A->...->Z | K=%d | Bufor=%d | pSleep=%dms |
                    cSleep=%dms | czas=%ds%n",
                 K, capacity, pSleepMs, cSleepMs, durationSeconds
        );

        System.out.println("\n--- STATYSTYKI ---");
        for (Producer p : producers) {
            System.out.printf("%s: produced=%d%n", p.getName(),
                p.getProduced());
            totalProduced += p.getProduced();
        }
        for (Stage s : stages) {
            System.out.printf("%s: processed=%d%n", s.getName(),
                s.getProcessed());
        }
        for (Consumer c : consumers) {
            System.out.printf("%s: consumed=%d%n", c.getName(),
                c.getConsumed());
            totalConsumed += c.getConsumed();
        }

        System.out.printf("SUMA: produced=%d, consumed=%d%n", totalProduced,
            totalConsumed);
    }
}
```

3 Wyniki działania programu

3.1 1 producent/1 konsument

```
Wariant: 1P = 1K  | Bufor=10 | pSleep=1ms | cSleep=1ms | czas=2s

—— STATYSTYKI ——
Producer-0: produced=994
Consumer-0: consumed=994
SUMA: produced=994, consumed=994
```

3.2 n_1 producentów / n_2 konsumentów ($n_1 > n_2$; $n_1 = n_2$; $n_1 < n_2$)

3.2.1 $n_1 > n_2$

```
Wariant: 9P > 6K  | Bufor=10 | pSleep=1ms | cSleep=1ms | czas=2s

—— STATYSTYKI ——
Producer-0: produced=615
Producer-1: produced=643
Producer-2: produced=678
Producer-3: produced=650
Producer-4: produced=617
Producer-5: produced=707
Producer-6: produced=631
Producer-7: produced=624
Producer-8: produced=589
Consumer-0: consumed=964
Consumer-1: consumed=954
Consumer-2: consumed=956
Consumer-3: consumed=971
Consumer-4: consumed=960
Consumer-5: consumed=949
SUMA: produced=5754, consumed=5754
```

3.2.2 $n_1 = n_2$

```
Wariant: 5P = 5K  | Bufor=10 | pSleep=1ms | cSleep=1ms | czas=2s

—— STATYSTYKI ——
Producer-0: produced=1012
Producer-1: produced=989
Producer-2: produced=986
Producer-3: produced=989
Producer-4: produced=980
Consumer-0: consumed=982
Consumer-1: consumed=978
Consumer-2: consumed=1000
Consumer-3: consumed=1005
Consumer-4: consumed=991
SUMA: produced=4956, consumed=4956
```

3.2.3 $n_1 < n_2$

```
Wariant: 5P < 7K  | Bufor=10 | pSleep=1ms | cSleep=1ms | czas=2s

—— STATYSTYKI ——
Producer-0: produced=956
Producer-1: produced=945
Producer-2: produced=944
Producer-3: produced=949
Producer-4: produced=934
Consumer-0: consumed=698
Consumer-1: consumed=669
Consumer-2: consumed=672
Consumer-3: consumed=687
Consumer-4: consumed=673
Consumer-5: consumed=663
Consumer-6: consumed=666
SUMA: produced=4728, consumed=4728
```

3.3 Przetwarzanie potokowe z buforem

```
POTOK A->...->Z | K=3 | Bufor=10 | pSleep=1ms | cSleep=1ms | czas=5s

—— STATYSTYKI ——
Producer-0: produced=1260
Stage-B: processed=1260
Stage-C: processed=1260
Stage-D: processed=1260
Consumer-0: consumed=630
Consumer-1: consumed=630
SUMA: produced=1260, consumed=1260

Process finished with exit code 0
```